

OFFENSIVE SECURITY PROJECT PLAN

Offensive Agents & Multi-Agent Penetration Testing Lab

Five-Week Build, Multi-Agent Exploitation & Reporting Plan

Project Duration: October 8 – November 12 (5-Week Plan) | **Team Size:** 5 Members

PROJECT TEAM & RECOMMENDED ROLES

Mohamed Zomor	Team Lead & System Architecture
Loay Eldesouky	Validation & False-Positive Elimination
Mohamed Abdelgawad	AI & Agent Orchestration
Nathan Nady	Security & MCP Tool Integration
Abdelrahman Reda	Bug Hunting & Reporting Sub-Agents

CONFIDENTIAL — ACADEMIC / LAB USE ONLY

1. Executive Summary

Our team builds an autonomous offensive security framework on OpenCode powered by multi-model LLMs as the central reasoning engine. A primary Operator Agent coordinates specialized sub-agents to automate the full penetration testing lifecycle: reconnaissance, Web/API vulnerability analysis, bug hunting, reporting, and false-positive elimination. All activities run against controlled targets to practice authorized assessment and bug bounty workflows.

1.1 Scope & Rules of Engagement

All automated testing runs exclusively inside authorized test environments and explicitly approved targets defined in `scope.json`. A technical interceptor blocks any out-of-scope requests before execution. Destructive actions, data tampering, and Denial-of-Service (DoS) payloads are completely disabled.

2. Technologies & Tools

Category	Tools & Technologies
AI & Reasoning Engine	Multi-Model LLM APIs (Gemini, Claude, GPT)
Agent Runtime & Orchestration	OpenCode Runtime, Primary Operator Agent
Tool Protocol (MCP)	Model Context Protocol (MCP CLI servers)
Storage & Evidence	SQLite (findings, targets, evidence, PoC logs)
Scanning & Traffic Analysis	Nmap, Burp Suite (Community / Pro API), Playwright
Interface & Versioning	React (live dashboard), Git / GitHub, Markdown

3. Project Workflow & Weekly Timeline (5-Week Plan)

Week 1 — Setup Environment: OpenCode, MCP Tools & API (Oct 8 – Oct 15)

- Initialize project repository and configure OpenCode runtime with multi-model LLM APIs.
- Set up and configure Model Context Protocol (MCP) tool connectors for external utilities.
- Implement technical proxy enforcement using `scope.json` to restrict target ranges.

Deliverable: *Operational OpenCode + Multi-Model LLM environment with MCP tools and active scope filter.*

Week 2 — Sub-Agent Workflow & Pentest Operations (Oct 16 – Oct 22)

- Design workflow architecture connecting Operator Agent to specialized sub-agents.
- Implement general pentest operational steps: reconnaissance, host mapping, and port scanning.
- Deploy authorized test targets and verify target asset mapping in SQLite.

Deliverable: *Automated Recon → Web assessment workflow operating in authorized test environment.*

Weeks 3 & 4 — Bug Hunting, Reporting & Mitigation Sub-Agent *(Oct 23 – Nov 5)*

- Design and build specialized Bug Hunting Sub-Agent targeting OWASP Top 10 vulnerabilities.
- Implement attack chaining, parameter inspection, and Burp Suite traffic replay mechanisms.
- Develop automated Reporting & Mitigation Sub-Agent to generate CVSS scores and code patches.

Deliverable: *Bug hunting engine with automated vulnerability reporting and mitigation generator.*

Week 5 — False-Positive Elimination Sub-Agent *(Nov 6 – Nov 12)*

- Design dedicated Validation Sub-Agent to systematically solve the false positive problem.
- Implement differential probing and out-of-band (OAST/OOB) verification mechanisms.
- Execute deterministic PoC scripts to mathematically confirm exploitability before logging findings.

Deliverable: *Active False-Positive Elimination Engine with verified PoCs and persistent SQLite store.*

4. Multi-Agent Security Methodology

The engagement follows a structured 5-phase automated workflow inspired by real-world assessments:

- **Reconnaissance & Mapping:** Identify active hosts, open ports, and API endpoints (Recon Agent, Nmap).
- **Vulnerability Auditing:** Inspect parameters and logic flows for OWASP Top 10 risks (Bug Hunting Agent, Burp).
- **False-Positive Validation:** Eliminate scanner noise through differential probing and deterministic PoC replay.
- **Reporting & Mitigation:** Generate executive summaries, CVSS scores, and developer code patches.
- **Continuous Governance:** Enforce scope.json and maintain tamper-evident audit logs across all steps.

5. Project Deliverables Matrix

Milestone Phase	Core Deliverable	Verification Standard
Week 1: Setup & Scope	OpenCode + Multi-Model Runtime, MCP Tools	100% rejection rate for out-of-scope targets
Week 2: Pentest Steps	Recon & Initial Web Assessment Workflow	Autonomous crawling in authorized test environment
Weeks 3 & 4: Bug Hunting	Hunting Engine & Automated Report Generator	Automated CVSS scoring and code patch output
Week 5: False Positives	Validation Engine & SQLite Persistence	Deterministic PoC verification without false alarms